

Desenvolvimento para Frontend

Introdução à TypeScript

Universidade Federal do Rio Grande do Norte

Módulos

- Para tornar elementos de um módulo disponíveis para outros módulos, usamos a palavra-chave **export**.

```
export class ZipCodeValidator {  
    isAcceptable(s: string) {  
        return s.length === 5;  
    }  
}  
  
export const numberRegex = /^[0-9]+$/;
```

- A classe ZipCodeValidator e a constante numberRegex são exportadas e podem ser usadas em outros módulos.

Módulos

- Para usar elementos de outros módulos, usa-se a palavra-chave `import`.

```
import { ZipCodeValidator } from "./ZipCodeValidator";  
let myValidator = new ZipCodeValidator();  
console.log(myValidator.isAcceptable("90210")); // true  
  
import * as validator from "./ZipCodeValidator";  
let myValidator2 = new validator.ZipCodeValidator();  
console.log(myValidator2.isAcceptable("12345")); // true
```

- Importações podem ser feitas de duas maneiras:
importação direta de elementos específicos ou importação de todo o módulo usando um **alias**.

Tipos Avançados - Tipos Literais

- Tipos literais permitem definir tipos que são representados por **valores específicos**, como strings, números ou booleanos.
- Em vez de aceitar qualquer valor, um tipo literal restringe a variável para valores **exatos e pré-definidos**.

Tipos Avançados - Tipos Literais

- `type`: Define um tipo personalizado chamado `Comando`.
 - `'start' | 'stop' | 'pause'`:
- Esses são tipos literais de string.
- O operador `|` representa uma união (union type), permitindo que `Comando` tenha um dos três valores possíveis.

```
type Comando = 'start' | 'stop' | 'pause';
```

Tipos Avançados - Union Types

- Union Types permitem que uma variável tenha um de vários tipos especificados, aumentando a flexibilidade.

```
function padLeft(value: string, padding: number | string) {  
  if (typeof padding === "number") {  
    return Array(padding + 1).join(" ") + value;  
  }  
  if (typeof padding === "string") {  
    return padding + value;  
  }  
  throw new Error(`Expected string or number, got '${typeof padding}'.`);  
}  
console.log(padLeft("Hello", 4)); // "    Hello"  
console.log(padLeft("Hello", "  ")); // "  Hello"
```

- Padding pode ser um number ou uma string, e o comportamento da função varia de acordo com o tipo de padding.

Tipos Avançados - Intersection Types

- Intersection Types permitem combinar múltiplos tipos em um, resultando em um tipo que tem todas as propriedades dos tipos combinados.

```
interface Person {  
  name: string;  
}  
  
interface Contact {  
  email: string;  
}  
  
type ContactInfo = Person & Contact;  
  
const contact: ContactInfo = {  
  name: "John Doe",  
  email: "john.doe@example.com"  
};  
console.log(contact.name); // "John Doe"  
console.log(contact.email); // "john.doe@example.com"
```

Tipos Avançados - Indexação de Tipo

- A indexação de tipo é um recurso poderoso no TypeScript que permite acessar tipos de propriedades específicas de um objeto ou array.
- Funciona de forma semelhante ao uso de colchetes ([]) em JavaScript para acessar valores de propriedades em tempo de execução, mas no TypeScript isso ocorre em tempo de compilação para manipulação de tipos.

Tipos Avançados - Indexação de Tipo

```
type TipoExtraido = Objeto["nomeDaPropriedade"];
```

- Objeto é um tipo que contém uma ou mais propriedades.
- "nomeDaPropriedade" é uma string literal que representa o nome da propriedade que queremos acessar.
- O TypeScript avalia e retorna o tipo correspondente àquela propriedade.

Tipos Avançados - Indexação de Tipo

```
type Usuario = {  
  nome: string;  
  idade: number;  
};  
  
type NomeDoUsuario = Usuario["nome"]; // string  
type IdadeDoUsuario = Usuario["idade"]; //number
```

- `Usuario["nome"]` acessa o tipo da propriedade nome do objeto `Usuario`, que é `string`.
- `Usuario["idade"]` acessa o tipo da propriedade idade, que é `number`.

Tipos Avançados – Condition Types

- Tipos Condicionais são expressões de tipo que selecionam um tipo com base em uma condição.

```
type MessageOf<T extends { message: unknown }> = T["message"];  
  
type Example1 = MessageOf<{ message: string }>; // string  
type Example2 = MessageOf<{ message: number }>; // number
```

- O tipo MessageOf seleciona o tipo da propriedade message de T.

Tipos Avançados – Condition Types

- Tipos Condicionais são expressões de tipo que selecionam um tipo com base em uma condição.

```
type Flatten<T> = T extends any[] ? T[number] : T;  
type Example3 = Flatten<string[]>; // string  
type Example4 = Flatten<number>;   // number
```

- O tipo Flatten verifica se T é um array e, se for, retorna o tipo dos elementos do array.

Classes Abstratas

- Em TypeScript, classes abstratas são classes que não podem ser instanciadas diretamente. Elas servem como modelos ou "contratos" para outras classes. Uma classe abstrata pode conter:
 - Métodos e propriedades concretos (implementados).
 - Métodos e propriedades abstratos (não implementados).
- Elas são úteis para definir uma estrutura base que outras classes devem seguir, garantindo consistência no design orientado a objetos.

Classes Abstratas

```
abstract class Forma {  
    abstract calcularArea(): number; // Método abstrato (sem implementação)  
  
    descricao(): void { // Método concreto (com implementação)  
        console.log("Esta é uma forma geométrica.");  
    }  
}
```

- Método Abstrato:
 - Não possui corpo (não tem implementação).
 - Deve ser implementado obrigatoriamente por qualquer classe que herde da classe abstrata.
- Método Concreto: Métodos comuns com implementação

Propriedades Abstratas

- Além de métodos, podemos também declarar propriedades abstratas, que devem ser implementadas pelas subclasses.

```
abstract class Pessoa {  
    abstract nome: string; // Propriedade abstrata  
  
    saudacao(): void {  
        console.log(`Olá, meu nome é ${this.nome}`);  
    }  
}  
class Estudante extends Pessoa {  
    nome: string; // Implementação da propriedade abstrata  
  
    constructor(nome: string) {  
        super();  
        this.nome = nome;  
    }  
}  
const estudante = new Estudante("João");  
estudante.saudacao(); // Olá, meu nome é João
```

Diferença entre Classes Abstratas e Interfaces

Aspecto	Classe Abstrata	Interface
Instanciação	Não pode ser instanciada	Não pode ser instanciada
Implementação	Pode ter métodos concretos e abstratos.	Só define contratos (sem lógica).
Uso de extends e implements	Subclasses herdam com extends.	Classes implementam interfaces.
Atributos/Propriedades	Pode conter atributos (variáveis).	Apenas declarações de atributos.
Herança Múltipla	Não suporta herança múltipla.	Suporta múltiplas interfaces.

Atividade 1

- Crie uma função que aceita um parâmetro de tipo literal que pode ser 'start', 'stop' ou 'pause'. Dependendo do valor, a função deve imprimir uma mensagem correspondente.

Atividade 1

- Crie uma função que aceita um parâmetro de tipo literal que pode ser 'start', 'stop' ou 'pause'. Dependendo do valor, a função deve imprimir uma mensagem correspondente.

```
type Comando = 'start' | 'stop' | 'pause';

function executarComando(comando: Comando): void {
  switch (comando) {
    case 'start':
      console.log('Iniciando...');
      break;
    case 'stop':
      console.log('Parando...');
      break;
    case 'pause':
      console.log('Pausando...');
      break;
  }
}

executarComando('start'); // Saída: Iniciando...
executarComando('stop');  // Saída: Parando...
executarComando('pause'); // Saída: Pausando...
```

Atividade 2

- Crie dois tipos, Endereco e Contato, e uma função que aceita um objeto que é a união desses dois tipos. A função deve imprimir todas as propriedades do objeto.

Atividade 2

- Crie dois tipos, Endereco e Contato, e uma função que aceita um objeto que é a união de classes distintas. A função deve imprimir todas as propriedades do objeto.

```
type Endereco = {  
  rua: string;  
  numero: number;  
};  
type Contato = {  
  telefone: string;  
  email: string;  
};  
type PessoaCompleta = Endereco & Contato;  
function imprimirPessoaCompleta(pessoa: PessoaCompleta): void {  
  console.log(`Rua: ${pessoa.rua}, Número: ${pessoa.numero}`);  
  console.log(`Telefone: ${pessoa.telefone}, Email: ${pessoa.email}`);  
}  
const pessoa: PessoaCompleta = {  
  rua: 'Rua Principal',  
  numero: 123,  
  telefone: '123-456-789',  
  email: 'pessoa@example.com'  
};  
imprimirPessoaCompleta(pessoa);
```

Atividade 3

- Crie uma classe abstrata Forma com um método abstrato calcularArea. Em seguida, crie duas classes que herdam de Forma: Circulo e Retângulo, implementando o método calcularArea para cada uma.

Atividade 3

- Crie uma classe abstrata `Forma` com um método abstrato `calcularArea`. Em seguida, crie duas classes que herdam de `Forma`: `Circulo` e `Retângulo`, implementando o método `calcularArea` para cada uma.

```
abstract class Forma {
    abstract calcularArea(): number;
}

class Circulo extends Forma {
    constructor(private raio: number) {
        super();
    }
    calcularArea(): number {
        return Math.PI * this.raio * this.raio;
    }
}

class Retangulo extends Forma {
    constructor(private largura: number, private altura: number) {
        super();
    }
    calcularArea(): number {
        return this.largura * this.altura;
    }
}

const circulo = new Circulo(5);
const retangulo = new Retangulo(10, 20);
console.log(circulo.calcularArea()); // Saída: 78.53981633974483
console.log(retangulo.calcularArea()); // Saída: 200
```

Referências

- <https://developer.mozilla.org/en-US/docs/Web/JavaScript/>
- <https://www.typescriptlang.org/docs/>
- <https://www.geeksforgeeks.org/>